hmi_port.c | 基础模块02_人机接口

源码完整导出 | V1.1 / 2026-10-09 | 各函数责任成员见注释

```c
/* 硬件适配模块：实现GD32 GPIO、硬件I2C1、ACK/超时检查和DWT延时。 */
#include "hmi_port.h"
#include "gd32f10x.h"
#define SCL GPIO_PIN_10
#define SDA GPIO_PIN_11
static uint8_t initialized;
/**
 * 功能：用DWT周期计数提供微秒延时
 * 参数：us：微秒数（内部仅1000）
 * 返回：无
 * 注意：SystemCoreClock必须准确，DWT需保持运行
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 delay_us。
 */
static void delay_us(uint32_t us)
{
    uint32_t start=DWT->CYCCNT;
    uint32_t cycles=(SystemCoreClock/1000000U)*us;
    while ((uint32_t)(DWT->CYCCNT-start)<cycles) {}
}
/**
 * 功能：配置或复位I2C1为100kHz主机。
 * 参数：无。返回：无。
 * 注意：复位清除外设状态，无法解除外部器件持续拉低的总线。
 * 作者/责任成员：吕朋聪；版本/日期：V1.1 / 2026-10-09。
 * 修改历史：2026-10-09，V1.1，首次实现 i2c_configure。
 */
static void i2c_configure(void)
{
    i2c_deinit(I2C1);
    i2c_clock_config(I2C1,100000U,I2C_DTCY_2);
    /* 本机地址仅用于外设配置，不是OLED从机地址。 */
    i2c_mode_addr_config(I2C1,I2C_I2CMODE_ENABLE,I2C_ADDFORMAT_7BITS,0x00U);
    i2c_enable(I2C1);
    i2c_ack_config(I2C1,I2C_ACK_ENABLE);
}
/**
 * 功能：等待硬件I2C标志，同时检查NACK、总线错误和仲裁丢失。
 * 参数：flag：目标标志；expected：SET或RESET。
 * 返回：0成功；-1错误或超时。
 * 注意：每阶段最长5ms，DWT需运行；不屏蔽中断，计数差支持回绕。
 * 作者/责任成员：吕朋聪；版本/日期：V1.1 / 2026-10-09。
 * 修改历史：2026-10-09，V1.1，首次实现 i2c_wait。
 */
static int i2c_wait(i2c_flag_enum flag, FlagStatus expected)
{
    uint32_t start=DWT->CYCCNT;
    uint32_t cycles=(SystemCoreClock/1000000U)*5000U;
    for (;;) {
        if (SET==i2c_flag_get(I2C1,I2C_FLAG_AERR) ||
            SET==i2c_flag_get(I2C1,I2C_FLAG_BERR) ||
            SET==i2c_flag_get(I2C1,I2C_FLAG_LOSTARB) ||
            SET==i2c_flag_get(I2C1,I2C_FLAG_OUERR)) return -1;
        if (expected==i2c_flag_get(I2C1,flag)) return 0;
        if ((uint32_t)(DWT->CYCCNT-start)>=cycles) return -1;
    }
}
/**
 * 功能：异常退出传输并复位I2C1状态。
 * 参数：无。返回：无。
 * 注意：只有仍为主机且未丢失仲裁时才发STOP；不重试原数据。
```

```
 * 作者/责任成员：吕朋聪；版本/日期：V1.1 / 2026-10-09。
 * 修改历史：2026-10-09，V1.1，首次实现 i2c_abort。
 */
static void i2c_abort(void)
{
    if (RESET==i2c_flag_get(I2C1,I2C_FLAG_LOSTARB) &&
        SET==i2c_flag_get(I2C1,I2C_FLAG_MASTER)) i2c_stop_on_bus(I2C1);
    i2c_configure();
}
/**
 * 功能：发送一个字节并等待完整传输及ACK结果。
 * 参数：value：待发字节。返回：0成功；-1错误或超时。
 * 注意：BTC表示移位器及数据寄存器完成；仅TBE不能证明收到ACK。
 * 作者/责任成员：吕朋聪；版本/日期：V1.1 / 2026-10-09。
 * 修改历史：2026-10-09，V1.1，首次实现 i2c_send_byte。
 */
static int i2c_send_byte(uint8_t value)
{
    if (i2c_wait(I2C_FLAG_TBE,SET)!=0) return -1;
    i2c_data_transmit(I2C1,value);
    return i2c_wait(I2C_FLAG_BTC,SET);
}
/**
 * 功能：初始化按键输入、100kHz硬件I2C1及DWT计时
 * 参数：无
 * 返回：无
 * 注意：幂等；先完成系统时钟初始化；保留PA0/PA1及SWD
 * 作者/责任成员：吕朋聪；版本/日期：V1.1 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_PortInit；V1.1，改用硬件I2C1。
 */
void HMI_PortInit(void)
{
    if (initialized) return;
    rcu_periph_clock_enable(RCU_GPIOA);
    rcu_periph_clock_enable(RCU_GPIOB);
    rcu_periph_clock_enable(RCU_AF);
    rcu_periph_clock_enable(RCU_I2C1);
    gpio_init(GPIOA,GPIO_MODE_IPU,GPIO_OSPEED_2MHZ,
              GPIO_PIN_5|GPIO_PIN_6|GPIO_PIN_7);
    gpio_init(GPIOB,GPIO_MODE_AF_OD,GPIO_OSPEED_50MHZ,SCL|SDA);
    CoreDebug->DEMCR|=CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CTRL|=DWT_CTRL_CYCCNTENA_Msk;
    i2c_configure(); initialized=1;
}
/**
 * 功能：读取三个功能键的瞬时按下状态
 * 参数：无
 * 返回：bit0~2对应SW1~SW3；1表示按下
 * 注意：内部上拉，未消抖；SW4为硬件复位不参与扫描
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_KeyRead。
 */
uint8_t HMI_KeyRead(void)
{
    uint8_t bits=0;
    if (RESET==gpio_input_bit_get(GPIOA,GPIO_PIN_5)) bits|=1;
    if (RESET==gpio_input_bit_get(GPIOA,GPIO_PIN_6)) bits|=2;
    if (RESET==gpio_input_bit_get(GPIOA,GPIO_PIN_7)) bits|=4;
    return bits;
}
/**
```

```
 * 功能：按毫秒执行短时阻塞延时
 * 参数：ms：延时毫秒数
 * 返回：无
 * 注意：仅OLED启动使用；需先初始化DWT；不关闭中断
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_DelayMs。
 */
void HMI_DelayMs(uint32_t ms)
{
    while (ms--) delay_us(1000);
}
/**
 * 功能：硬件I2C1轮询写SSD1306，检查NACK、总线错误和5ms阶段超时
 * 参数：control：0x00命令/0x40数据；data：数据；len：1~128字节
 * 返回：0成功；-1总线超时/NACK/非法参数
 * 注意：仅主循环；PB10/PB11复用开漏，需外部3.3V上拉；失败复位外设，不重试
 * 作者/责任成员：吕朋聪；版本/日期：V1.1 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_OLEDWrite；V1.1，改用硬件I2C1。
 */
int HMI_OLEDWrite(uint8_t control, const uint8_t *data, uint16_t len)
{
    uint16_t i;
    if (!initialized || data==0 || len==0 || len>128 ||
        (control!=0x00 && control!=0x40)) return -1;
    if (i2c_wait(I2C_FLAG_I2CBSY,RESET)!=0) goto fail;
    i2c_start_on_bus(I2C1);
    if (i2c_wait(I2C_FLAG_SBSEND,SET)!=0) goto fail;
    /* 库函数要求左移一位的地址；0x3C对应写地址0x78。 */
    i2c_master_addressing(I2C1,(uint32_t)(HMI_OLED_ADDRESS<<1),I2C_TRANSMITTER);
    if (i2c_wait(I2C_FLAG_ADDSEND,SET)!=0) goto fail;
    i2c_flag_clear(I2C1,I2C_FLAG_ADDSEND);
    if (i2c_send_byte(control)!=0) goto fail;
    for (i=0;i<len;++i) {
        if (i2c_send_byte(data[i])!=0) goto fail;
    }
    i2c_stop_on_bus(I2C1);
    if (i2c_wait(I2C_FLAG_I2CBSY,RESET)!=0) goto fail;
    return 0;
fail:
    i2c_abort(); return -1;
}
/**
 * 功能：保存中断屏蔽状态并进入短临界区
 * 参数：无
 * 返回：原PRIMASK值
 * 注意：仅保护取键值，不在临界区进行OLED传输
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_EnterCritical。
 */
uint32_t HMI_EnterCritical(void)
{
    uint32_t mask=__get_PRIMASK(); __disable_irq(); return mask;
}
/**
 * 功能：恢复进入临界区前的中断状态
 * 参数：mask：EnterCritical返回值
 * 返回：无
 * 注意：必须与EnterCritical成对使用
 * 作者/责任成员：吕朋聪；版本/日期：V1.0 / 2026-10-09。
 * 修改历史：2026-10-09，V1.0，首次实现 HMI_ExitCritical。
 */
```

```
184 void HMI_ExitCritical(uint32_t mask)
185 {
186     __set_PRIMASK(mask);
187 }
```